HostDiag: Towards Accurate and Low-interference Intra-host Bottlenecks Diagnostics for Cross-platform RDMA Networks

Paper # 222, 12 pages body, 13 pages total

ABSTRACT

Distributed Machine Learning (DML) services are sensitive to intra-host failures and performance bottlenecks, which become more common as AI accelerator platforms diversify. In addition, some software-level misconfigurations behave like hardware bottlenecks and can waste troubleshooting time. However, existing mechanisms cannot quickly detect and locate intra-host problems across heterogeneous platforms or determine whether the service problem is host-related.

In this paper, we propose HostDiag, the first platform-agnostic intra-host monitoring and diagnostic system based on cross-layer active probing. HostDiag can accurately measure one-way link performance based on various AI accelerators (GPUs and NPUs), distinguish between task-induced interference and hardware faults, and judge whether a performance drop is host-related and assess its impact on services. We have deployed the full implementation of HostDiag on Huawei Ascend NPUs and a prototype on NVIDIA devices in large-scale clusters. Evaluation results show that HostDiag can detect, categorize, and locate intra-host bottlenecks in seconds with high accuracy across different hardware architectures. HostDiag efficiently detects and locates 13 types of problems during deployment, and we share our experience in building a consistent diagnostic view for the post-GPU era.

1 INTRODUCTION

AI accelerators, including GPUs, NPUs, and various specialized ASICs, have been deployed in large-scale data centers to meet the surging computational demands of modern distributed machine learning (DML) [2, 4, 8, 21, 27]. While NVIDIA CUDA was long considered the de-facto standard for AI computing, the industry is shifting toward a diverse compute landscape. Modern DML clusters typically select different AI accelerator platforms according to task-specific requirements, such as Huawei Ascend NPUs and Google TPUs, to achieve better energy efficiency and lower Total Cost of Ownership (TCO) in large-scale inference and training tasks [1, 8, 12, 22, 28].

Timely and accurately locating intra-host performance bottlenecks is critical to ensuring the stable operation of DML services. The performance of DML tasks exhibits *The Barrel Effect*, making them extremely sensitive to intra-host anomalies. For example, a 50% degradation in PCIe bandwidth on a single node can severely reduce the training throughput of the entire cluster. Some hidden faults, such as degraded PCIe links or misconfigured high-speed interconnects (e.g., NVLink [25] or HCCS [10]), do not trigger explicit failures but cause gray failures that lead to persistent performance regressions. As the size of diverse accelerator networks grows from hundreds to tens of thousands of units, the frequency and impact of these intra-host problems are increasing.

In addition, when service performance degrades, operators must quickly determine if the problem is network-related, host-related, or application-related. This is not easy. First, the shift away from CUDA-specific hardware has not been matched by the evolution of monitoring tools. Most existing diagnostic systems are deeply coupled with the NVIDIA driver stack and specific hardware topologies, creating blind spots on non-CUDA platforms. Second, many intra-host anomalies are gray failures, including PCIe link degradation and interconnects defects, which do not trigger explicit errors but cause severe performance regressions. Furthermore, the root causes of these bottlenecks are weakly correlated with observable performance metrics, such as increased execution time or throughput drops, are highly similar to performance fluctuations caused by improper DML configurations or scheduling mechanisms. Without a cross-platform framework, operators often waste significant time manually comparing logs and metrics to distinguish between hardware bottlenecks and software-level fluctuations.

However, existing methods cannot quickly detect and locate intra-host bottlenecks in cross-platform deployments. Previously, the method of diagnosing host performance problems relied on vendor-specific profilers or manual inspection of hardware counters. While tools like Hostping [17] have explored intra-host probing, they are limited by their dependency on specific driver APIs and their inability to perform accurate one-way measurement. As DML clusters scale and diversify, these troubleshooting steps become increasingly time-consuming, and their inability to distinguish between task interference and hardware faults leads to inaccurate diagnoses.

In this paper, we extend Hostping [17] and present HostDiag, the first cross-platform intra-host monitoring and diagnostic system based on cross-layer active probing, to address the above problems. To fully meet our requirements, we need to achieve four design targets that cannot be achieved by existing tools. First, we need to provide a unified abstraction to support diverse AI accelerators (GPUs, NPUs, and ASICs) regardless of their proprietary drivers. Second, we need to accurately measure one-way intra-host link performance to locate specific bottlenecks. Third, we need to distinguish between DML task-induced fluctuations and real hardware faults to minimize interference. Finally, we need to assess the impact of these problems on services using a unified schema, enabling seamless correlation with data center network (DCN) management system.

First, HostDiag establishes a unified abstraction to identify key APIs for non-intrusive monitoring and probing by analyzing deepl learning (DL) and AI accelerator libraries. Second, HostDiag identifies precise timing to accurately measure one-way bandwidth and latency across PCIe and high-speed interconnects by utilizing `memcpy` for active probing. Third, HostDiag distinguishes between transient task fluctuations and hardware bottlenecks by analyzing monitoring and probing data. It categorizes identified bottlenecks into tiered levels (P0–P2) to accurately assess their actual impact on service performance. Finally, by adopting the Unified Fault Telemetry Schema (UFTS), HostDiag aggregates these diagnostic results into a standardized format, enabling seamless correlation with DCN management system for cross-layer root-cause analysis.

We have deployed the full implementation of HostDiag on the Huawei Ascend NPU platform and a prototype system on NVIDIA devices to verify its cross-platform portability. During deployment, HostDiag can detect, categorize, and locate intra-host bottlenecks in seconds and determine their impact on services. Our evaluation shows that HostDiag effectively identifies 13 types of problems caused by hardware aging, driver defects, and configuration errors. By correlating host-level data with network telemetry, we significantly reduce the complexity of fault attribution. We share our experience in dealing with these problems, such as identifying PCIe bandwidth drops during model checkpoint loading. In summary, this paper makes the following contributions:

- We present the first cross-platform intra-host monitoring and diagnostic system. It can be easily deployed in cross-platform DML clusters with low overhead and provides a unified observability semantic across GPUs and NPUs.
- HostDiag innovatively proposes (1) an extened Berkeley Packet Filter (eBPF)-based monitor to track DML throughput across different DL libraries, (2) a task-aware prober to measure one-way link performance with minimal interference and maximum accuracy, and (3) the UFTS for cross-layer fault correlation.
- We summarize the hidden intra-host faults detected by HostDiag in our experiments and share our experiences in building a consistent diagnostic view for next-generation DML clusters with diverse AI accelerator platforms.

2 BACKGROUND AND MOTIVATION

2.1 Is it still CUDA?

Over the past decade, **CUDA has been almost synonymous with AI computing**. Leveraging mature programming models, stable driver stacks, and a comprehensive software ecosystem, NVIDIA GPUs have become the de-facto standard for AI infrastructure. Consequently, many system designs including scheduling, Collective Communication Libraries (CCLs), intra-host monitoring and diagnostic mechanisms,rest on the implicit premise that *the AI accelerators are CUDA GPUs.*

As computational demands evolve, long-standing system assumptions face unprecedented pressure. As AI applications shift from training toward large-scale online inference and continuous deployment, the focus has changed significantly. The sensitivity of inference to energy consumption and cost has exposed the limitations of general-purpose GPUs in energy efficiency and operational expenses. Consequently, a *one-size-fits-all approach relying solely on a single architecture is no longer feasible.*

As demand shifts toward high-efficiency inference, **NPUs, TPUs, and specialized ASICs are emerging as critical platforms beyond GPUs**. Represented by platforms such as Huawei Ascend, NPUs employ specialized architectures tailored for neural network operators, achieving a **35%–70% energy efficiency improvement** under typical inference workloads [8]. This provides a clear TCO advantage for power-constrained, long-running clusters. Similarly, Google TPUs [15] enhance performance-per-dollar through matrix-calculation units and high-bandwidth interconnects. Specialized ASICs tailored for DML applications further improve latency and deployment density by narrowing hardware functional boundaries. Consequently, modern DML clusters now consist of diverse AI accelerator platforms rather than single-platform CUDA architectures [12, 21, 28].

Unfortunately, **while computing platforms have begun moving beyond CUDA GPUs, fault management systems have not kept pace.** Different AI accelerator platforms are highly heterogeneous in driver interfaces and interconnects architectures. However, a large number of intra-host monitoring and diagnostic tools are still designed based on NVIDIA GPU system conditions and are deeply bound

to the CUDA driver stack. For instance, probing and diagnostics mechanisms of the most representative intra-host probing work, Hostping [17], are designed specifically for NVIDIA Remote Direct Memory Access (RDMA) Network Interface Cards (RNICs) and GPUs. By relying on proprietary APIs and assuming NVIDIA-specific topologies, such systems struggle to adapt to other architectures or be reused directly on non-CUDA platforms.

Once removed from the NVIDIA GPU environment, these design assumptions quickly fail, creating severe blind spots within other DML clusters. When inference latency rises or training throughput drops, existing systems can provide fine-grained intra-host diagnostic results on NVIDIA devices. In contrast, on non-NVIDIA AI accelerator platforms, anomalies can often only be coarsely labeled as `host-side problems`. It is difficult to further determine whether the root cause lies in the AI accelerator, the driver layer, or the internal intra-host links, making it impossible to form a unified performance understanding and root-cause analysis at a cross-platform level.

Therefore, **the question is no longer whether to use CUDA, but whether the system still assumes everything is CUDA-based.** To truly support the diverse AI accelerator platforms for next-generation DML clusters, intra-host fault management mechanisms must break free from dependence on a single vendor's ecosystem and instead build a cross-platform, scalable monitoring and diagnostic framework that can:

- Run uniformly across GPUs, NPUs, and other AI accelerators.
- Abstract hardware and driver differences behind a unified interface.
- Provide consistent observability semantics for DCN management system.

With non-NVIDIA devices becoming first-class citizens, there is a need for a tool that rethinks the design of intra-host active probing and fault management.

2.2 Hidden and Harmful Intra-host Faults

In large-scale distributed systems, the most harmful faults are often not explicit failures but persistent, elusive performance degradations. Similar to gray failures in networks, intra-host hardware and driver-layer anomalies typically do not trigger explicit errors or device offline events and instead cause continuous degradation of DML task performance, even though the system remains operational at a significantly reduced speed.

It occurs more frequently on newer AI accelerator platforms. Compared with the well-established NVIDIA CUDA ecosystem, these platforms are still evolving in driver implementations, device management, and interconnects configurations, making them more susceptible to hidden performance anomalies. We have repeatedly observed several types of intra-host hidden faults in production DML clusters, including:

- **PCIe Performance Degradation**: Hardware aging can degrade PCIe link quality, leading to reduced bandwidth or increased latency and, in turn, lowering DMA efficiency.
- **Interconnects Anomalies**: Misconfigurations or driver defects can make high-speed interconnects unavailable, forcing traffic to fall back to PCIe, causing congestion and reduced energy efficiency.

Local performance degradation triggered by faults within a single host can be amplified in a distributed system, causing prolonged performance regression at the system level. In production, we observed that a 50% reduction in PCIe bandwidth of a single host could potentially cause an instantaneous decrease in the training speed of the entire DML cluster by more than 50%. More importantly, without explicit system failures, such degradation is frequently dismissed as normal fluctuation and left undiagnosed, leading to sustained degradation of overall performance.

As AI accelerators become increasingly diverse, promptly detecting and quantifying hidden intra-host performance degradations is critical for ensuring large-scale DML cluster stability.

2.3 Hard to Detect and Locate Bottlenecks

Beyond explicit intra-host faults, persistent bottlenecks pose an additional challenge to the performance of DML clusters. Similar to hidden faults, these bottlenecks do not result in task termination, yet they subtly degrade performance over time.

A frequent example in our clusters is **PCIe bandwidth degradation of NPU**. Since large data transfers between host DDR and NPU HBM memory depend on PCIe links, these links can lose over 45% of their bandwidth due to power, thermal, and driver constraints, resulting in at least an 80% increase in task execution time and slower cluster training.

Detecting such bottlenecks is challenging because existing monitoring systems cannot provide effective intra-host metrics, due to the following reasons:

- **Limited granularity**: Existing monitoring systems expose only round-trip bandwidth and latency for entire paths, which prevents precise identification of bottlenecks or isolation of per-flow throughput.
- **Task interference**: Active probing shares bandwidth with DML tasks, making it difficult to identify the cause of bandwidth degradation.

- **Ambiguous symptoms**: Increased task execution time or reduced throughput is hard to distinguish from workload fluctuations, misconfigurations, or intra-host bottlenecks.

These challenges demand a low-interference, fine-grained intra-host monitoring tool capable of cross-layer correlation to achieve accurate understanding of intra-host state.

2.4 Targets & Limitations of Hostping [17]

We need an efficient, low-interference, and cross-platform intra-host monitoring and diagnostic system to promptly detect and locate performance degradation and resource bottlenecks. Such a system should provide consistent and correlatable observability semantics for DCN management system. When DML training or inference performance degrades, it is crucial to quickly determine whether the root cause lies within the host. Hostping [17], a representative intra-host active probing system, pioneered the use of loopback tests to locate intra-host bottlenecks on NVIDIA servers. *Can Hostping [17] be directly extended to support today's diverse accelerator platforms?* Unfortunately, although an important foundation is built by Hostping [17], its core system assumptions are difficult to generalize to modern DML clusters with GPUs, NPUs, and specialized ASICs. We summarize our design targets and analyze Hostping's [17] limitations as follows.

(1) Cross-platform intra-host fault diagnostics. Hostping's [17] tightly coupled probing and analysis architecture limits its applicability to cross-platform hardware. Different vendors expose proprietary monitoring interfaces and driver models, while Hostping [17] relies on hard-coded, NVIDIA-specific metrics and diagnostic logic. Supporting a new platform requires rewriting both the probing module and analysis engine. As a result, Hostping [17] cannot be easily extended to non-NVIDIA platforms. For example, Huawei Ascend NPUs lack hardware counterparts to NVIDIA RNICs assumed by Hostping [17].

(2) Correlating intra-host and inter-host faults. Hostping [17] detects intra-host anomalies but reports them in isolation, without integration with inter-host monitoring systems such as R-Pingmesh [16]. This separation prevents cross-layer correlation and validation between intra-host and inter-host symptoms, significantly reducing root-cause localization accuracy. For instance, increased DML training latency may result from PCIe bottlenecks or AI accelerator overload within the host, or from network congestion and packet reordering. Without correlation, operators must manually compare host logs and network telemetry, which often takes tens of minutes or more.

(3) Accurate one-way link measurement under task interference. Precisely locating intra-host bottlenecks requires quantifying one-way link performance and distinguishing hardware faults from DML workload fluctuations. However, Hostping's [17] loopback tests only provide end-to-end round-trip metrics, making it difficult to identify bottleneck links. During busy periods, Hostping [17] can detect intra-host bottlenecks but can only infer the most likely faulty link. Moreover, lacking awareness of DML task behavior, Hostping [17] cannot determine whether anomalies stem from task workload or system errors.

(4) Impact-aware evaluation of intra-host issues. Not all detected anomalies have equal impact on DML tasks. Some of them, such as transient spikes in PCIe latency, may have limited effect, while others require immediate action. Consistent strategies risk over-intervention, including unnecessary resource reallocation or disruptive operations such as server restarts. Therefore, a tiered evaluation mechanism based on task impact is necessary:

- **P0 (Critical):** Hardware-level faults (e.g., PCIe disconnection or AI accelerator interconnects failure) that fatally impact DML tasks and require immediate handling.
- **P1 (Important):** System-level gray failures (e.g., long-term bandwidth or latency degradation) that pose risks in future and require cost-aware repair decisions.
- **P2 (Warning):** Task-level interference (e.g., bandwidth contention or misconfigurations) where guidance is preferred over intrusive actions.

This tiered evaluation enables precise root-cause localization and differentiated responses. When DML throughput drops without P0 or P1 faults, troubleshooting can focus on task logic. Hostping [17] lacks such task-aware severity modeling, making it difficult for operators to distinguish critical hardware faults from transient performance interference.

3 HOSTDIAG OVERVIEW

3.1 Challenges and Opportunities

To achieve efficient and scalable intra-host fault management across diverse AI accelerator platforms, we address three main challenges:

Cross-platform intra-host abstraction. To support deployment across NVIDIA, Huawei, and other accelerators, monitoring must avoid vendor-specific interfaces. Modern platforms differ in driver models, performance counters, and interconnects architectures. Existing tools tightly couple probing with hardware, resulting in high migration costs and inconsistent metrics. The challenge is constructing a unified monitoring abstraction from fragmented low-level APIs, addressed in Section 4.1.1 (monitoring layer) and Section 4.2.1 (probing layer).

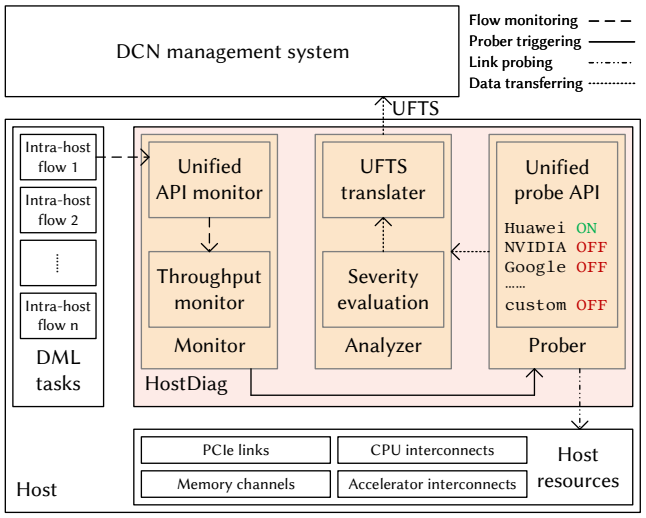

Figure 1: HostDiag framework and its key components.

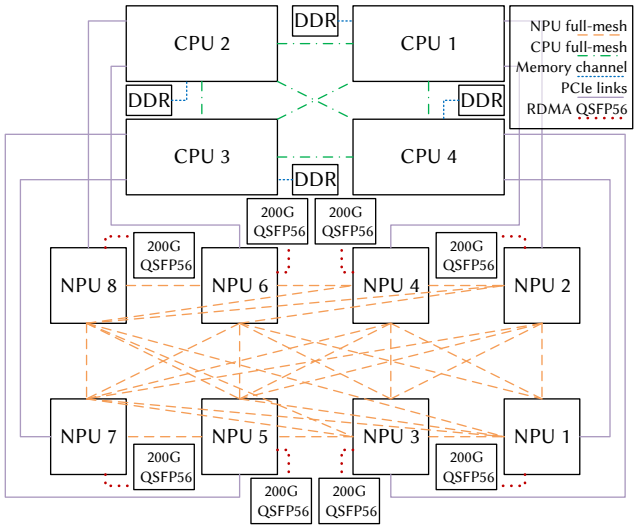

Figure 2: Topo of Huawei Ascend 910B platform and its main path for communication. HostDiag use it as a basic reference to design cross-platform monitoring and probing.

Decoupling one-way link bottlenecks under adaptive workloads. Evaluating link performance requires separating DML task traffic from link bottlenecks. Tools based on eBPF can monitor function calls to get DML task. However, the challenge is that it cannot directly capture DML traffic, which complicates task state assessment and probing window selection. We address this in Section 4.1.2 (window selection) and Section 4.2.2 (one-way measurement).

Cross-layer fault attribution and task impact evaluation. First, a severity-based evaluation system is needed to assess bottleneck severity and plan differentiated responses. Thus, the first challenge is to accurately evaluate the severity of anomalies, which we address in Section 4.3.1 (severity-based evaluation). Furthermore, intra-host anomalies must be correlated with network alarms and task execution to identify root causes. For example, when DML task throughput drops sharply, the system must distinguish whether the root cause is from PCIe bottlenecks, network congestion, or application misconfigurations. Accurate fault attribution poses another key challenge, as it requires cross-layer correlation and a unified fault representation, which we address in Section 4.3.2 (UFTS).

3.2 HostDiag Framework

Figure 1 shows HostDiag's architecture, a cross-platform, cross-layer intra-host fault management system. Monitoring and probing are decoupled from diagnosis and localization, enabling cross-platform adaptation. Diagnostic results use the UFTS for reporting to the DCN management system, enabling end-to-end observability.

HostDiag runs on RDMA hosts and has three core modules including *monitor*, *prober*, and *analyzer*. *Monitor* based on eBPF hooks key operations in DL and AI accelerators libraries to collect path, size, and timestamp in data transferring. On detecting abnormal throughput, the *prober* selects a window based on anomaly severity and host load, then injects micro-probe packets tailored to the accelerator to measure one-way bandwidth and latency. The *analyzer* localizes bottlenecks and assesses anomaly severity, expressing results via UFTS for cross-layer correlation. HostDiag provides intra-host bottleneck localization and service fault attribution. Figure 2 illustrates the core components of the Huawei Ascend 910B platform and its topology. We choose this topology as the design baseline because it has the most simplified structure. Other platforms, such as NVIDIA AI accelerator platforms, can extend this topology by adding RNICs. Therefore, the probing logic proposed here can be naturally extended to those platforms as well.

The objective of intra-host bottleneck link localization is to monitor the performance status of links within DML cluster hosts to promptly discover and locate performance bottlenecks. HostDiag implements an eBPF-based *monitor* to collect DML throughput of intra-host network. When abnormal degradation of throughput is detected, the system selects an appropriate window to trigger the *prober* for one-way link measurement. The probing packets injected by the *prober* are similar in form to the DML task traffic copies and are subject to the same intra-host bottlenecks as the task traffic. Consequently, the *prober* traverses the same path as

Table 1: Feature comparisons of monitoring methods

Features	Real-time View	Profiler	eBPF Based
Monitor All Metrics	✗	✓	✓
Non-invasive Design	✓	✗	✓

the abnormal DML traffic along its path, measuring one-way bandwidth and latency to reveal the true state of the intra-host links and accurately locate the specific bottleneck.

Fault attribution further analyzes the impact of intra-host link anomalies on DML tasks, building upon bottleneck link localization. The *analyzer* determines the fault severity level based on measurement results from the abnormal intra-host links. Irrecoverable intra-host bottlenecks are assigned a P0 fault alert. For bottlenecks that can recover autonomously or exhibit persistent fluctuations, the *analyzer* tracks them over a period and assigns a P0 or P1 alert based on the duration of the anomaly and its actual impact on DML tasks. Additionally, the *analyzer* records and reports performance degradation caused by DML task misconfigurations or scheduling mechanisms, categorizing these as P2 warnings to assist users in troubleshooting task-level issues. After completing the fault level assessment, the *analyzer* expresses the results via the UFTS and reports them to the DCN management system for comprehensive evaluation and processing. HostDiag achieves precise attribution of intra-host link anomalies and enables cross-layer analysis.

4 DESIGN

4.1 HostDiag Monitor

4.1.1 Select Suitable Hook Points for Multiple DL Libraries

The primary consideration in designing the *monitor* for various DL libraries is the choice of monitoring method. We have two core requirements: (1) accurately measuring DML throughput, and (2) employing a universal monitoring logic across DL libraries such as PyTorch and MindSpore. Table 1 compares three monitoring methods.

Accurate DML throughput measurement requires five key pieces of information: (1) *Size* of sent data, (2) sender node N_{send}, (3) receiver node N_{recv}, (4) flow start time t_{start}, and (5) flow end time t_{end}. Throughput is then calculated via Equation (1).

$$Th_{N_{send} \to N_{recv}} = \frac{Size}{t_{end} - t_{start}} \tag{1}$$

Since DL and AI accelerators libraries do not directly expose this information, only partial data can be obtained through analysis, which limits our ability to fully assess the system performance.

Table 2: List of APIs by AI accelerator libraries

Type	CUDA API [23]	Ascend API [9]
Set Dev	`cudaSetDevice`	`aclrtSetDevice`
Alloc Mem	`cudaMallocHost`	`aclrtMallocHost`
	`cudaMallocDevice`	`aclrtMallocDevice`
Mem Cpy	`cudaMemcpy`$^{\triangle}$	`aclrtMemcpy`$^{\triangle}$
	`cudaMemcpyAsync`*	`aclrtMemcpyAsync`*
	`cudaMemcpy2D`	`aclrtMemcpy2d`
	`cudaMemcpy2DAsync`*	`aclrtMemcpy2dAsync`*
Free Mem	`cudaFreeHost`	`aclrtFreeHost`
	`cudaFree`	`aclrtFree`
Sync Strm	`cudaStreamSynchronize`	`aclrtSynchronizeStream`

($\triangle$) indicates basic memory copy functions.
($*$) indicates asynchronous memory copy functions.

Regarding the three monitoring methods, feasibility depends on whether they can obtain all five key pieces. *Real-time view* programs like `top` or `htop` only capture overall resource occupancy. Estimating data size or flow timestamps from resource usage is inaccurate and requires complex calculations. More importantly, system-level monitoring cannot identify traffic nodes, making it insufficient for fine-grained DML monitoring. In contrast, the other two methods can track DML call paths and packet sizes, and using the timestamps of function call start and end, obtaining all five pieces accurately.

Profiler tools from DL libraries or accelerators (e.g., PyTorch Profiler, NVIDIA CUDA Profiling Tools) obtain fine-grained information via deep integration, but require inserting Profiler APIs into application code, which is intrusive and unsuitable for large-scale production. *Real-time* view and *eBPF-based* monitoring avoid application modifications, enabling low-intrusive monitoring.

Combining these insights, *real-time view* programs fail to meet fine-grained DML monitoring needs, and *profiler* tools are intrusive. The *eBPF-based* monitoring selectively hooks functions in kernel or user space, capturing all necessary information without modifying application code.

The eBPF monitoring workflow is as follows. Despite variations in high-level DL communication interfaces, all ultimately call accelerator runtime APIs (e.g., CUDA Runtime, AscendCL) for data transfers. Therefore, we monitor at the runtime interface layer. Analysis of different runtimes shows communication operations follow similar call patterns and naming conventions (Table 2). Based on these patterns, the universal workflow is:

(1) Obtain device IDs N_{send} and N_{recv} via *device setting* functions; use system defaults if not set.

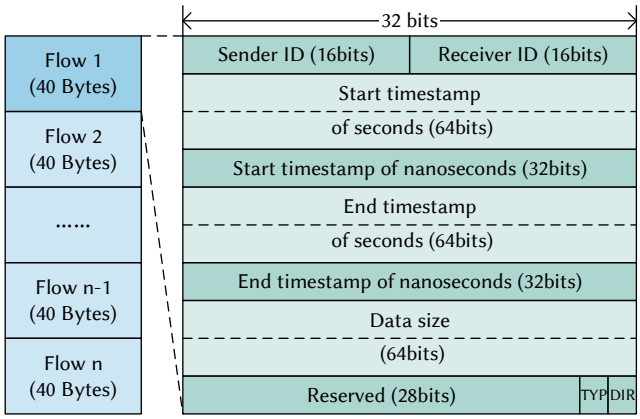

Figure 3: Flow-path records in binary format.

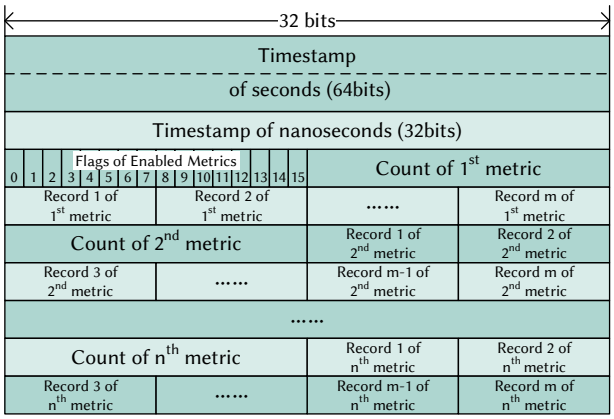

Figure 4: Intra-host resource usage in binary format.

(2) Capture recent *memory allocation* calls and map allocated addresses to device IDs.
(3) Record timestamp t_{start} when a *memory copy* function is invoked.
(4) Retrieve *memory copy* arguments to record *Size*, and query device IDs from source/destination addresses.
(5) For synchronous calls, record t_{end} on return; for asynchronous calls, monitor corresponding *stream synchronization* function return.
(6) Capture *resource release* calls and remove address-to-device mappings.

A custom configuration interface allows users to specify which functions to monitor, enabling adaptive support for different DL libraries and accelerator types.

4.1.2 *Measure DML Throughput with eBPF*

To measure DML task throughput with eBPF, we use `pid` and `tid` to distinguish throughput from each sub-thread under multi-process or multi-thread conditions. For threads with the same `pid` and `tid`, communication call sequences are captured via the workflow in Section 4.1.1. For multi-threaded tasks with different `pid`/`tid`, we first partition data by sender and receiver device IDs, then accumulate throughput of each thread between its start and end timestamps. The instantaneous throughput of DML traffic through link l_0 at time t_0 is calculated as:

$$Th_{l=l_0}|_{t=t_0} = \sum_{\substack{i,j \\ i\neq j, l_0 \in p(N_i,N_j)}} Th_{N_i \to N_j}|_{t=t_0} \quad (2)$$

Where $l_0 \in p(N_i, N_j)$ indicates that the path from N_i to N_j contains link l_0, and the set of all such paths is:

$$P_{l_0} = \{p(N_i, N_j) | l_0 \in p(N_i, N_j)\} \quad (3)$$

All links and paths are directed.

After obtaining instantaneous throughput, link utilization is computed as the ratio to its theoretical bandwidth, $Util_{l=l_0}|_{t=t_0}$. When $Util_{l=l_0}|_{t=t_0} > Util_{high}$, the link is busy and requires no probing. If DML task is in communication phase but $Util_{low} < Util_{l=l_0}|_{t=t_0} < Util_{high}$, a potential performance anomaly is indicated, triggering active probing to distinguish between task-induced fluctuations and inherent link bottlenecks.

For platforms with independent RNICs (e.g., NVIDIA), we also monitor abnormal RNIC metrics such as PFC pause frame counts and transmit queue lengths to guide probing. For platforms with integrated RDMA (e.g., Huawei Ascend), we focus on throughput variations of AI accelerator memory copy functions.

To assess DML service operation, the *monitor* periodically collects intra-host resource usage, including CPU, host memory, AI accelerator, and accelerator HBM utilization, enabling correlation analysis between link anomalies and task workloads. The *monitor* generates files for the *prober* and *analyzer*:

- Device-ID mapping file in CSV format.
- Flow-path records file in binary format Figure 3, containing the five key pieces of information from Section 4.1.1; additionally, there is a set of 32-bit identifier flags and reserved space, with the final 4 bits representing the traffic type and traffic direction.
- Intra-host resource usage time-series file in binary format Figure 4.

4.2 HostDiag Prober

4.2.1 *Probe Across Different AI Accelerator Platforms*

Similar to the *monitor*, the *prober* has two core objectives: (1) precisely measure one-way link bandwidth and latency while excluding DML task throughput; and (2) achieve cross-platform adaptability across AI accelerator platforms. Based

on these goals and intra-host communication characteristics, major intra-host links are categorized into two types for separate measurement.

PCIe and CPU interconnects buses. These links connect CPU, memory, RNIC, and AI accelerators. They lack non-blocking point-to-point topology and are subject to contention. Hostping [17] used RDMA verbs for loopback tests, but lacks universality and accurate one-way measurement. We extend this approach to meet cross-platform adaptation and one-way measurement by refining two test types:

(a) *PCIe links connected to RNICs*: we adopt Hostping's [17] loopback method to measure RNIC-to-memory performance, then estimate one-way bandwidth and latency using metrics from other links.
(b) *Other PCIe links and CPU interconnects buses*: we use one-way measurement via memory copy functions, accurately capturing bandwidth and latency across all paths.

Architectures with separate GPUs and RNICs components (e.g., NVIDIA) need both methods for more accurate metrics. On devices integrating computing and networking (e.g., Huawei Ascend), method (b) suffices. Since method (b) interfaces follow similar call patterns in different platforms, it is the primary method for cross-platform adaptation. Implementation details for PCIe and CPU interconnects measurements are in Section 4.2.2.

Full-mesh interconnects links between AI accelerators. These high-speed links (e.g., NVLink for NVIDIA, HCCS for Ascend) feature non-blocking topologies without contention. When anomalies occur, we invoke `all-to-all` test samples from the accelerator's CCL via `mpirun` to measure bandwidth and latency. Detection and invocation interfaces automatically select the appropriate tests per platform.

Probing logic is modularized into a cross-platform plugin system, dynamically loaded during HostDiag initialization based on the host's AI accelerator type. When abnormal throughput is detected, the *prober* invokes the module to inject probes, measure one-way bandwidth and latency, and report results in a unified format. Adapting to a new platform requires only mapping memory copy functions and interconnects test interfaces, integrating device capabilities while reducing engineering costs for cross-platform intra-host fault management.

4.2.2 Measure One-way Bandwidth & Latency with Memcpy

We use synchronous memory copy functions listed in Table 2 to design a one-way intra-host link measurement method. Unlike Hostping [17], by leveraging link throughput data from the *monitor* (Section 4.1), we accurately assess current link load, avoiding continuous full-scale probing. When service traffic is normal, the *prober* performs no measurements, preventing interference with DML tasks. Upon detecting an abnormal throughput drop, the *prober* targets only paths associated with the abnormal link, minimizing overhead while localizing bottlenecks. For example, if a specific PCIe path between an AI accelerator and host shows abnormal throughput, probe packets are injected only for paths between this accelerator and all regions of host memory. If these paths perform normally, the link is excluded as a bottleneck; otherwise, a proportion of paths to other host memory channels are randomly selected to exclude memory channel issues. The workflow is:

(1) Obtain instantaneous utilization $Util_{l=l_0}|_{t=t_0}$ and DML task state of link l_0 at t_0 to decide whether to trigger measurement.
(2) Retrieve all paths $p(N_i, N_j)$ in P_{l_0} and sequentially perform memory copy measurements, recording $bw_{N_i \to N_j}$ and $lat_{N_i \to N_j}$.
(3) Determine if each path is abnormal ($bw < bw_{th}$ or $lat > lat_{th}$). If abnormal paths exceed $ratio_{th}$, continue probing; otherwise, terminate.
(4) Randomly select a proportion $ratio_{ran}$ of unmeasured paths p_{sample}, perform measurements, and record the results of $bw_{N_k \to N_m}$ and $lat_{N_k \to N_m}$.
(5) If all of p_{sample} are normal, link l_0 is the bottleneck; otherwise, a large-scale intra-host bottleneck is indicated, requiring full measurement.

Memory copy measurements use AI accelerator library functions: `MallocHost` for host memory and `MallocDevice` for device memory to allocate buffers of size $Size$ at N_{send} and N_{recv}. Start timestamp t_{start} is recorded before copying via synchronous `Memcpy`, and t_{end} upon completion. Latency uses small probe packets ($lat = t_{end} - t_{start}$), and bandwidth uses large packets (Equation (1)). After measurement, `FreeHost` and `Free` release memory. We employ a probing approach similar to that used by the monitored functions list in Table 2.

On platforms with separate RNICs, a Hostping [17]-based loopback test measures RNIC-to-host memory paths. Since other paths are measured in one-way, this helps identify abnormal links more accurately.

The probing modules output results in a unified probe-path records file, which is similar to the *monitor*'s flow-path records binary file (Figure 3). It stores the five key pieces of information (Section 4.1.1) and shares the device-ID mapping CSV file. RNIC loopback results are marked with the direction (DIR) bits in Figure 3.

4.3 HostDiag Analyzer

4.3.1 Assess the Impact of Host-level Bottlenecks

The *analyzer* ingests the flow-path files and resource usage time-series files generated by the *monitor* in Section 4.1.2,

alongside the probe-path files produced by the *prober* in Section 4.2.2, to localize intra-host bottleneck links. Based on these sequence files, the *analyzer* can recalculate the bandwidth for any intra-host path at any moment and retrieve latency data obtained from small-packet probing. By leveraging the correlation between different paths and controlling variables among them, the *analyzer* accurately identifies the specific locations of intra-host bottleneck links. These bottleneck links are characterized by bandwidth below a preset threshold bw_{th} or latency exceeding the threshold lat_{th}.

Following localization, the *analyzer* evaluates the severity of these intra-host link anomalies. The primary assessment metrics include:

(a) Bandwidth below the threshold bw_{th};
(b) Latency above the threshold lat_{th};
(c) Anomaly duration exceeding the threshold $time_d$;
(d) Anomaly time ratio exceeding the threshold $time_r$;
(e) Proportion of abnormal links exceeding the threshold $link_r$;
(f) Abnormal drop in DML task throughput.

Based on these metrics, the *analyzer* follows a tiered assessment process to classify severity:

(1) If a link satisfies at least one metric from (a) or (b), and also satisfies at least one metric from (c)–(e), it is identified as a critical intra-host bottleneck link causing severe failure and is assigned a P0 fault alert.
(2) If a link satisfies at least one metric from (a) or (b), but fails to meet any metrics from (c)–(e), it is categorized as an important intra-host bottleneck link and assigned a P1 fault alert.
(3) If a link satisfies none of the metrics from (a)–(e) but triggers metric (f), the performance degradation is likely caused by DML task configurations or scheduling mechanisms, resulting in a P2 warning alert.

For P0 and P1 level faults, the final collected data is forwarded to the DCN management system for unified processing and analysis. For P2 alerts, the *analyzer* reports the recorded data to the application side to facilitate troubleshooting of task-level performance issues.

4.3.2 *Aggregate Fault Telemetry Through Unified Schema*

Upon completing the processing of raw data, the *analyzer* represents the results using the UFTS and reports them to the DCN management system to enable cross-layer fault correlation and root-cause analysis. Specifically, the data reporting is organized into two layers: fault event data and fault metric data.

Fault event data describes the analyzed fault information through the encoded fields of the UFTS, shown in Figure 5.

Following each `fault_device_id` field, several groups of `fault_metric` fields are appended. The exact count of them

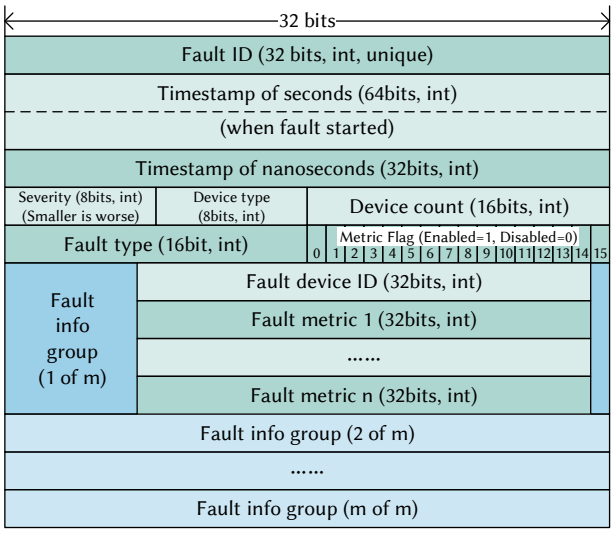

Figure 5: UFTS in binary format. A fault info group is a set of binary data that contains a `fault_device_id` and several `fault_metrics`.

is determined by the preceding `metric_flag` field, in which each bit represents a specific metric type. If a bit is set to 1, the corresponding metric value is included in the sequence. For instance, if the 0th, 2nd, and 5th bits of the `metric_flag` are set to 1, it indicates that three sets of `fault_metric` values follow the `fault_device_id`. Each `fault_device_id` field and its corresponding `fault_metric` values collectively describe multiple metrics of a single faulty device, forming a basic fault description group. The number of such groups within each fault event data packet is defined by the `device_count` field.

After the DCN management system receives this data, if further cross-comparison analysis is required, it can request the analyzer to provide more detailed fault metric data, which contains the raw data captured by the monitor and prober. Upon receiving a request from the management platform, the system retrieves data from the corresponding time period within the flow-path files, probe-path files, and resource usage time-series files to construct fault metric data in the following format:

- `fault_id`: A 32-bit unsigned integer that serves as the unique identifier corresponding to the fault event data.
- `reserved`: A 64-bit field of all-zero data used to differentiate fault metric data from fault event data.
- `data_type`: A 32-bit unsigned integer indicating the type of data file being transmitted.
- `raw_data`: The raw recorded records of the fault data.

Both fault event data and fault metric data use binary encoding for transmission to reduce overhead and improve

system responsiveness. Additionally, the 64-bit second-level portion of the timestamp is utilized to distinguish between the transmission of fault event data and fault metric data, thereby preventing data confusion.

5 IMPLEMENTATION

We have developed the full implementation of HostDiag on Huawei Ascend 910A and 910B Series NPU platforms. These platforms use Kunpeng 920 CPUs with the AArch64 architecture and run different operating systems, including EulerOS and Ubuntu. In addition, we implemented a prototype on x86-based NVIDIA GPU servers to validate the cross-platform portability of HostDiag.

The HostDiag *monitor* uses `bpfcc` to track intra-host DML traffic throughput. We obtain platform information via `lspci`, then locate the AI accelerator runtime libraries based on environment variables for hooking, thereby capturing the API calls listed in Table 2. We set $Util_{high}$ to 80% to determine whether link throughput is normal, and set $Util_{low}$ to 1% on PCIe 4.0 hosts to filter out data transfers outside communication phases.

The HostDiag *prober* is triggered by the *monitor* to perform active probing, measuring the bandwidth and latency of suspected abnormal links to determine whether bottlenecks exist. We set bw_{th} to 80% of baseline bandwidth and lat_{th} to 150% of baseline latency to identify bandwidth degradation or latency inflation on intra-host links. For the abnormal path $ratio_{th}$, we set 50% for PCIe links of AI accelerators and 25% for other high-speed links. To validate the detected bottleneck links, we randomly sample 25% of links for verification, i.e., $ratio_{ran} = 25\%$.

The HostDiag *analyzer* reads the binary probe records produced by the *monitor* and *prober* to conduct severity-based evaluation. The sustained fault duration threshold $time_d$ is set to 5 minutes, the fault time ratio threshold $time_r$ is set to more than 20% within the past hour, and the large-scale faulty-link ratio $link_r$ is set to 25%. If a bottleneck first satisfies the P1 criteria, the *analyzer* marks it as P1 and reports it, then continuously tracks its status in subsequent monitoring to determine whether to upgrade it to P0.

6 EVALUATION & BOTTLENECKS FOUND

We deployed HostDiag on our experimental platform for more than one month. Figure 2 shows the topology of the core components in the Ascend 910B NPU testbed. Each server has four full-mesh connected Kunpeng 920 CPUs. Each CPU connects to memory on two NUMA nodes and two 910B NPUs. Every two NPUs also provide two 200 Gbps RoCE ports. Each server contains eight full-mesh connected Ascend 910B NPUs. For Ascend 910A, each server has two

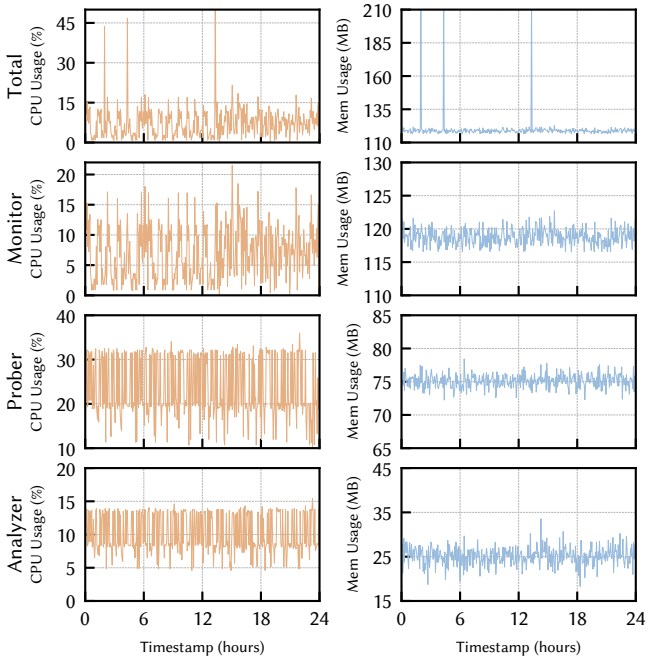

Figure 6: CPU and memory usage of HostDiag in a day. The CPU usage is based on one core of Kunpeng 920.

independent NPU full-mesh fabrics, each connecting four 910A devices.

HostDiag overhead. We design modular *monitor*, *prober*, and *analyzer* components to enable cross-platform adaptability. This introduces modest overhead that is acceptable. Figure 6 shows HostDiag's CPU and memory usage, with average overheads of 7.34% CPU and 119.54 MB memory. Only the *monitor* runs continuously, consuming 7.05% CPU and 118.79 MB memory. The *prober* and *analyzer* run on demand. To assess their worst-case impact, we ran each component continuously and measured system overhead; the *prober* consumed 23.39% CPU and 75.11 MB memory, while the *analyzer* consumed 10.06% CPU and 25.12 MB memory.

For a server with 192 CPU cores and 768 GB memory, these overheads are negligible. Our experimental results also show that DML tasks do not always fully saturate all resources, so the overhead has little impact on training time. In tests that run the same DML workload with HostDiag enabled and disabled, the average runtime across multiple tests remains consistent.

HostDiag accuracy. Over one month of test deployment, we observed 43 intra-host problems of varying severity. There are 15 hardware bottlenecks and 28 DML task misconfigurations. The hardware bottlenecks were verified to stem from host stability issues, and each incident lasted no more than 10 minutes. For DML task misconfigurations, some were due to configuration mistakes during training, while others were

Table 3: Problems found by HostDiag on Huawei Ascend platform during our testing deployment.

No	Problems categories	Root cause	Typical Detecting Methods
1	System faults	NPU PCIe links BW DEC	The PCIe path BW from any host memory to NPU has DEC.
2		Memory channels BW DEC	Memory channel BW from any NPU has DEC.
3		CPU interconnects BW DEC	Data transfer BW between NUMA nodes has DEC.
4		NPU interconnects DISABLED	There is only throughput in PCIe but not in NPU interconnects.
5	Performance bottlenecks	NPU PCIe links BW/LAT JIT	PCIe path BW/LAT JIT fast.
6		Memory channels BW/LAT JIT	Memory channel BW/LAT JIT fast.
7		CPU interconnects BW/LAT JIT	Data transfer BW/LAT between NUMA nodes JIT FAST.
8		Cross-NUMA data loading	NPU loads checkpoints from memory in other NUMA nodes.
9		SSD/HDD R/W bottleneck	Checkpoint loads/saves in low speed.
10	DML tasks misconfigurations	Inappropriate parallel parameter	Low NPU or HBM usage.
11		Inappropriate batch size	A large number of small data flows appears. / Slow single-flow but high total throughput in loading.
12		Inappropriate device selection	There is significant difference in HBM usage on different devices.
13		Frequent checkpoint saving	High NPU-to-DDR traffic with low NPU utilization.

* BW: Bandwidth; LAT: Latency; DEC: Decrease; INC: Increase; JIT: Jitter.

induced by our simulated misconfigurations. HostDiag effectively distinguishes among problem types and issues tiered alerts based on severity.

When triggering the *prober*, the *monitor* provides an appropriate probing window, but interference from bursty traffic cannot be fully ruled out. We therefore tested the false-positive rate when running the *prober* alone. Based on time proportion, the false-alarm rate is about 1.3%. However, in the full HostDiag system, we have not observed such false alarms, because the *analyzer* jointly evaluates monitoring data from the *monitor* and probe data from the *prober*, improving overall diagnostic accuracy.

Typical problems found by HostDiag. Using HostDiag, we identify three typical classes of intra-host issues: *system faults*, *performance bottlenecks*, and *DML task misconfigurations*. *System faults* usually have a severe impact on DML tasks and persist for a long duration, and are typically classified as P0. Faults #1–#3 are often caused by hardware, drivers, or the operating system. Fault #4 is typically due to misconfiguration or driver anomalies that disable high-speed interconnects. These faults significantly affect DML tasks and require immediate remediation.

Performance bottlenecks can also severely impact DML tasks, but the impact depends on the fraction of time the bottleneck occurs. Bandwidth or latency jitter on intra-host links is common but does not always affect DML tasks. Therefore, we continuously track these issues and classify them as P0 or P1 based on the final impact assessment in Section 4.3.1. Faults #5–#7 exhibit symptoms similar to #1–#3, but usually include additional latency fluctuations and larger bandwidth variance. Fault #8 is a common issue caused by incorrect NUMA binding or allocation. This bottleneck typically affects only checkpoint loading/saving and does not directly slow training, but it increases checkpoint I/O time. Fault #9 is similar to #8 and also impacts checkpoint I/O. However, it is often constrained by hardware and is difficult to resolve quickly.

DML task misconfigurations originate at the application level. DML tasks must be re-tuned for different AI accelerator platforms to maximize performance. This tuning often requires experienced developers to understand new hardware characteristics and to perform careful planning before training begins, which is unfavorable for fast-evolving Large Language Model (LLM) workloads. For emerging AI accelerator platforms, performance characteristics also require validation through real deployments. With HostDiag probing, we gain a detailed view of DML execution and its bottlenecks, helping developers improve task performance while helping accelerator vendors understand real resource usage and further optimize their devices. Faults #10–#13 are not only misconfigurations but also indicate requirements for future AI accelerator design.

7 RELATED WORK

Intra-host Network and RNICs Bottleneck Diagnosis. With the rapid increase in RDMA network line speed in recent years, end-side bottlenecks have gained significant attention in research literature[13, 14, 17–19, 31, 33, 34]. Here,

we introduce the work most relevant to HostDiag. Hostping [17] actively probes intra-host link latency and bandwidth to discover intra-host network bottlenecks. EMOGI [20] integrates FPGAs into intra-host links and employs soft switches to monitor intra-host traffic. Lumina [33] reveals correctness testing methods for RDMA hardware network stacks. Collie [14] utilizes search algorithms to uncover potential performance bottlenecks in RNICs. HostDiag is the first monitoring and diagnostic work to systematically correlate intra-host DML task throughput with link bottlenecks. Furthermore, it provides cross-platform adaptation for various DL libraries and AI accelerator platforms.

DCN Bottleneck and Anomaly Diagnosis. Extensive research and tools exist regarding DCN monitoring and diagnosis, which can be categorized into network-oriented and server-oriented tools based on their detection purposes[5, 6, 16, 29, 35, 36]. Network-oriented tools aim to identify performance bottlenecks throughout the DCN. ByteTracker [6] implements agentless DCN monitoring for millisecond-level fault diagnosis. R-Pingmesh [16] extends Pingmesh [5] to RoCE networks to achieve service-aware probing. Conversely, server-oriented tools focus on revealing intra-host performance bottlenecks. NVIDIA Mellanox NEO [24] provides diagnostic counter functions for NVIDIA RNICs, while NVIDIA SMI [26] provides real-time status for NVIDIA GPUs. Huawei MindCluster ToolBox [11] offers full-system performance testing for Atlas servers, and Huawei NPU-SMI [10] provides real-time status for Ascend NPUs. HostDiag is the first tool designed to build a unified framework that merges network-oriented and server-oriented bottleneck diagnosis. By expressing DCN monitoring data through the UFTS, it achieves fused diagnosis of intra-host and inter-host network monitoring data.

Fault Detection and Root Cause Analysis. As the demand for network infrastructure from LLMs increases, accurate network fault detection and root cause analysis have become critical research directions[3, 7, 30, 32]. Aegis[3] concurrently collects intra-host and inter-host metrics to locate AI training faults via correlation analysis. Murphy[7] utilizes Markov Random Fields to model loose dependencies across virtual machines, containers, and hosts in distributed applications for performance root-cause inference. BiAN[30] leverages LLM models to analyze network device alarm texts for fault localization assistance. NetLLM[32] explores adapting LLMs to network tasks such as viewport prediction and bitrate adaptation. HostDiag is the first system to achieve decoupling of probing and analysis pipelines through modular construction. This enables high-precision intra-host bottleneck link localization and provides formatted data for cross-layer root causes analysis in other systems.

8 CONCLUSION

In this paper, we propose HostDiag, the first cross-platform intra-host monitoring and diagnostic system based on cross-layer active probing. It is based on task-aware design and unified abstraction interface, which can be easily deployed in cross-platform DML clusters with low overhead. HostDiag helps accurately detect hardware failures, locate performance bottlenecks, and analyze root causes through fault correlation. HostDiag can effectively distinguish between task-induced fluctuations and hardware-level gray failures while assessing their impact on services. We have deployed and evaluated HostDiag on multiple platforms to demonstrate its effectiveness, and it has become an essential framework for ensuring the performance stability of our DML clusters. *This work does not raise any ethical issues.*

REFERENCES

[1] Marco Armoni. 2024. Tensor processing units (TPU): A technical analysis and their impact on artificial intelligence. *URL https://tech4future. info/wp-content/uploads/2024/11/Tensor-Processing-Units-TPU-Paper-ENG. pdf* (2024).

[2] ByteBridge. [n. d.]. GPU and TPU Comparative Analysis Report. ([n. d.]). https://bytebridge.medium.com/gpu-and-tpu-comparative-analysis-report-a5268e4f0d2a

[3] Jianbo Dong, Kun Qian, Pengcheng Zhang, Zhilong Zheng, Liang Chen, Fei Feng, Yichi Xu, Yikai Zhu, Gang Lu, Xue Li, et al. 2025. Evolution of Aegis: Fault Diagnosis for AI Model Training Service in Production. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 865–881.

[4] Hira Ehtesham. [n. d.]. AI Chip Market Statistics: The $118B Boom Reshaping Semiconductors. ([n. d.]). https://www.allaboutai.com/resources/ai-statistics/ai-chip-market

[5] Chuanxiong Guo, Lihua Yuan, Dong Xiang, Yingnong Dang, Ray Huang, Dave Maltz, Zhaoyi Liu, Vin Wang, Bin Pang, Hua Chen, et al. 2015. Pingmesh: A large-scale system for data center network latency measurement and analysis. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. 139–152.

[6] Shixian Guo, Kefei Liu, Yulin Lai, Yangyang Bai, Ziwei Zhao, Songlin Liu, Jianghang Ning, Gen Li, Jianwei Hu, Yongbin Dong, et al. 2025. ByteTracker: An Agentless and Real-time Path-aware Network Probing System. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 541–553.

[7] Vipul Harsh, Wenxuan Zhou, Sachin Ashok, Radhika Niranjan Mysore, Brighten Godfrey, and Sujata Banerjee. 2023. Murphy: Performance diagnosis of distributed cloud applications. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 438–451.

[8] Youngpyo Hong and Dongsoo Kim. 2025. Performance and efficiency gains of npu-based servers over gpus for ai model inference. *Systems* 13, 9 (2025), 797.

[9] Huawei. [n. d.]. Ascend C library. ([n. d.]). https://www.hiascend.com/document/detail/zh/CANNCommunityEdition/850/opdevg/Ascendcopdevg/atlas_ascendc_map_10_0002.html

[10] Huawei. [n. d.]. Huawei Ascend HDK. ([n. d.]). https://www.hiascend.com/hardware/firmware-drivers/community

[11] Huawei. [n. d.]. MindCluster. ([n. d.]). https://www.hiascend.com/document/detail/zh/mindcluster/730/toolbox/toolboxug/toolboxug_0002.html

[12] Shiho Kim and Ganesh Chandra Deka. 2021. *Hardware accelerator systems for artificial intelligence and machine learning*. Vol. 122. Academic Press.

[13] Xinhao Kong, Jingrong Chen, Wei Bai, Yechen Xu, Mahmoud Elhaddad, Shachar Raindel, Jitendra Padhye, Alvin R Lebeck, and Danyang Zhuo. 2023. Understanding RDMA microarchitecture resources for performance isolation. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 31–48.

[14] Xinhao Kong, Yibo Zhu, Huaping Zhou, Zhuo Jiang, Jianxi Ye, Chuanxiong Guo, and Danyang Zhuo. 2022. Collie: Finding performance anomalies in RDMA subsystems. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 287–305.

[15] Peidong Lin, Hexu Zhang, Lin Li, and Yun Li. 2024. Systolic Array Architecture for Multitasking Neural Processing Unit. In *2024 29th International Conference on Automation and Computing (ICAC)*. IEEE, 1–6.

[16] Kefei Liu, Zhuo Jiang, Jiao Zhang, Shixian Guo, Xuan Zhang, Yangyang Bai, Yongbin Dong, Feng Luo, Zhang Zhang, Lei Wang, et al. 2024. R-pingmesh: A service-aware roce network monitoring and diagnostic system. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 554–567.

[17] Kefei Liu, Zhuo Jiang, Jiao Zhang, Haoran Wei, Xiaolong Zhong, Lizhuang Tan, Tian Pan, and Tao Huang. 2023. Hostping: Diagnosing intra-host network bottlenecks in RDMA servers. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 15–29.

[18] Jiaqi Lou, Xinhao Kong, Jinghan Huang, Wei Bai, Nam Sung Kim, and Danyang Zhuo. 2024. Harmonic: Hardware-assisted RDMA Performance Isolation for Public Clouds. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1479–1496.

[19] Teng Ma, Tao Ma, Zhuo Song, Jingxuan Li, Huaixin Chang, Kang Chen, Hai Jiang, and Yongwei Wu. 2019. X-RDMA: Effective RDMA middleware in large-scale production environments. In *2019 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 1–12.

[20] Seung Won Min. 2022. *Fine-grained memory access over I/O interconnect for efficient remote sparse data access*. Ph.D. Dissertation. University of Illinois at Urbana-Champaign.

[21] Ashutosh Mishra, Jaekwang Cha, Hyunbin Park, and Shiho Kim. 2023. *Artificial intelligence and hardware accelerators*. Springer.

[22] Pothukuchi Nikhila. 2025. Understanding Tensor Processing Units: The Specialized Hardware Revolutionizing AI Computing. (2025).

[23] NVIDIA. [n. d.]. CUDA Toolkit Documentation 13.1 Update 1. ([n. d.]). https://docs.nvidia.com/cuda/index.html

[24] NVIDIA. [n. d.]. NVIDIA Mellanox Management Software. ([n. d.]). https://www.nvidia.com/en-us/networking/management-software/

[25] NVIDIA. [n. d.]. NVIDIA NVLink and NVLink Switch. ([n. d.]). https://www.nvidia.com/en-us/data-center/nvlink/

[26] NVIDIA. [n. d.]. NVIDIA System Management Interface SMI. ([n. d.]). https://developer.nvidia.com/system-management-interface

[27] Grand View Research. [n. d.]. AI Accelerator Market (2025 - 2033). ([n. d.]). https://www.grandviewresearch.com/industry-analysis/ai-accelerator-market-report

[28] Cristina Silvano, Daniele Ielmini, Fabrizio Ferrandi, Leandro Fiorin, Serena Curzel, Luca Benini, Francesco Conti, Angelo Garofalo, Cristian Zambelli, Enrico Calore, et al. 2025. A survey on deep learning hardware accelerators for heterogeneous hpc platforms. *Comput. Surveys* 57, 11 (2025), 1–39.

[29] Cheng Tan, Ze Jin, Chuanxiong Guo, Tianrong Zhang, Haitao Wu, Karl Deng, Dongming Bi, and Dong Xiang. 2019. NetBouncer: Active device and link failure localization in data center networks. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. 599–614.

[30] Chenxu Wang, Xumiao Zhang, Runwei Lu, Xianshang Lin, Xuan Zeng, Xinlei Zhang, Zhe An, Gongwei Wu, Jiaqi Gao, Chen Tian, et al. 2025. Towards llm-based failure localization in production-scale networks. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 496–511.

[31] Shicheng Wang, Menghao Zhang, Xiao Li, Qiyang Peng, Haoyuan Yu, Zhiliang Wang, Mingwei Xu, Xiaohe Hu, Jiahai Yang, and Xingang Shi. 2025. Hawkeye: Diagnosing rdma network performance anomalies with pfc provenance. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 481–495.

[32] Duo Wu, Xianda Wang, Yaqi Qiao, Zhi Wang, Junchen Jiang, Shuguang Cui, and Fangxin Wang. 2024. Netllm: Adapting large language models for networking. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 661–678.

[33] Zhuolong Yu, Bowen Su, Wei Bai, Shachar Raindel, Vladimir Braverman, and Xin Jin. 2023. Understanding the micro-behaviors of hardware offloaded network stacks with lumina. In *Proceedings of the ACM SIGCOMM 2023 Conference*. 1074–1087.

[34] Yiwen Zhang, Yue Tan, Brent Stephens, and Mosharaf Chowdhury. 2022. Justitia: Software Multi-Tenancy in Hardware Kernel-Bypass Networks. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 1307–1326.

[35] Hao Zheng, Chengyuan Huang, Xiangyu Han, Jiaqi Zheng, Xiaoliang Wang, Chen Tian, Wanchun Dou, and Guihai Chen. 2024. μMon: Empowering Microsecond-level Network Monitoring with Wavelets. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 274–290.

[36] Yibo Zhu, Nanxi Kang, Jiaxin Cao, Albert Greenberg, Guohan Lu, Ratul Mahajan, Dave Maltz, Lihua Yuan, Ming Zhang, Ben Y Zhao, et al. 2015. Packet-level telemetry in large datacenter networks. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. 479–491.